Snowflake Interim Tag Based Masking Policy Code

Introduction

This page highlights the creation of the following objects to support an implementation of the interim masking approach.

- Snowflake Tags
- Snowflake Masking Policies

The interim masking approach is closely coupled to the interim classification process and capture.

Classifications are captured here.

Masking policy rules logic

The classification process informs whether an attribute is 'maskable', this logic applies:

IF ROLE ACCESS allows the user to see the data attribute unmasked

THEN RETURN the attribute value unmasked

ELSE the above is not true so we test if the attribute is deemed 'Maskable' via the following rules

IF the **security classification** HAS a value of either 'Confidential' OR 'High Confidential'

AND if **any of** the following flag values **is 'Y'** (true)

privacy_identifier_flag
pci_dss_compliance_identifier_flag
government_identifier_flag
sensitive_personal_identifier_flag
health_identifier_flag
credit_identifier_flag
credentials_identifer_flag

THEN APPLY MASK and **RETURN** the attribute value masked (e.g. '**MASKED**)

ELSE RETURN the attribute value unmasked as none of the above is true and thus the attribute in question does not require masking

Tag and Policy Creation and implementation

We will create the PoC code in an existing BDH 'tenancy' (which in future state will alter to subject area zones), and use the development environment for 'Customer').

NOTE: The roles and snowflake databases and schemas here are for PoC purposes only. For future state implementation these will be different.

STEP 1 - CREATE THE REQUIRED SNOWFLAKE TAGS SPECIFIC FOR ATTRIBUTE SENSITIVITY CLASSIFICATION

First, we must set the user to a role which has tag object creation privileges:

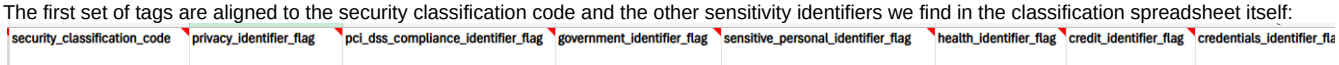
Use a role which has TAG object permissions

```
-- use customer tenancy in BDH dev environment for interim masking PoC
USE DATABASE DB_CUSTOMER_DEV02;


-- use a role which has tag object privileges
USE ROLE RL_CUSTOMER_DEV02_ADMIN;
```

We then create relevant tags which match those recorded in the interim classification process.

The first set of tags are aligned to the security classification code and the other sensitivity identifiers we find in the classification spreadsheet itself:



We add these tags in a 'Governance' schema, but **NOTE**, for future state we advise that these are created in a separate Subject Area Zone (i.e. not Customer specific), such as an Enterprise Data Governance/Utility zone so the tags are accessible to all other subject area zones.

Create Tags to match those recorded in the interim classification process

```
-- Create relevant tags in Snowflake which match those recorded in the interim classification processes
-- Tags to add (Name [Values]):
---- security_classification_code [Private/BNZ Internal, Public, Confidential, Highly Confidential)
---- privacy_identifier_flag [Y,N]
---- pci_dss_compliance_identifier_flag [Y,N]
---- government_identifier_flag [Y,N]
---- sensitive_personal_identifier_flag [Y,N]
---- health_identifier_flag [Y,N]
---- credit_identifier_flag [Y,N]
---- credentials_identifer_flag [Y,N]

-- In future state we should look to create these tags in the utility zone - for BDH PoC we create them in
the Customer Tenancy
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code ALLOWED_VALUES 'Private/BNZ
Internal', 'Public', 'Confidential', 'Highly Confidential';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag ALLOWED_VALUES 'Y', 'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag ALLOWED_VALUES 'Y',
'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag ALLOWED_VALUES 'Y', 'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag ALLOWED_VALUES 'Y',
'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag ALLOWED_VALUES 'Y', 'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag ALLOWED_VALUES 'Y', 'N';
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag ALLOWED_VALUES 'Y', 'N';

-- Show the tags created
SHOW TAGS IN SCHEMA DB_CUSTOMER_DEV02.GOVERNANCE;
```

STEP 2 - CREATE MASKING POLICIES

We now create masking policies. Masking policies in Snowflake must receive and output the same datatype.

We create three different masking policies as we are testing the mask for three datatypes:

- String
- Number
- Date

Create Masking Policies

```
-- we can create one masking policy per datatype.  Each of the following masking policies will be assigned
to a single tag which is applied to a column.  The tag then calls the relevant masking policy depending on
the attribute's datatype.
-- we will use the role 'RL_DAS_DEV02_CONSUMER' as the role which ALLOWS the attribute to be seen unmasked
(data is shown in it's original form)
-- if the user does not have the role 'RL_DAS_DEV02_CONSUMER' then data will be shown as masked if the
conditional attribute sensitivity code block determines the attribute should be masked.
-- NOTE: the use of the role 'RL_DAS_DEV02_CONSUMER' is for test only as roles which map to functional
groups (which have been defined access levels) have not yet been created.

-- for number datatype the mask is to present the attribute's value as '-1'
CREATE OR REPLACE MASKING POLICY DB_CUSTOMER_DEV02.GOVERNANCE.SENSITIVE_DATA_MASK_NUMBER AS (VAL number)
RETURNS Number ->
CASE
    WHEN
        // Check to see if the role(s) which has been identified as the one which ALLOWS the data attribute
to be presented UNMASKED
        // Replace 'RL_DAS_DEV02_CONSUMER' with the actual role (TBD within the uplift/data product streams)
        IS_ROLE_IN_SESSION('RL_DAS_DEV02_CONSUMER')
        THEN VAL -- return the unmasked value, exit policy here
    WHEN
        // The following code block determines whether the attribute passed to the policy should be subject
to masking or not
        // If it meets the following criteria then it is required that the attribute needs masking
        (
            // The security classification MUST be either 'Confidential' or 'Highly Confidential'
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code')
in ('Confidential', 'Highly Confidential')
            AND
```

```
            // At least one of the following classification flags must be 'Y'
            (
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
pci_dss_compliance_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag')
= 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
sensitive_personal_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag')
= 'Y'
            )
        )
        THEN -1 -- return the masked value, exit policy here, the user DOES NOT have the data role to allow
unmasked data AND the attribute qualifies for masking
    ELSE VAL -- return the attribute value unmasked as none of the above is true and thus the attribute in
question does not require masking
END;

-- for String datatype the mask is to present the attribute's value as '**MASKED**'
CREATE OR REPLACE MASKING POLICY DB_CUSTOMER_DEV02.GOVERNANCE.SENSITIVE_DATA_MASK_STRING AS (VAL string)
RETURNS String ->
CASE
    WHEN
        // Check to see if the role(s) which has been identified as the one which ALLOWS the data attribute
to be presented UNMASKED
        // Replace 'RL_DAS_DEV02_CONSUMER' with the actual role (TBD within the uplift/data product streams)
        IS_ROLE_IN_SESSION('RL_DAS_DEV02_CONSUMER')
        THEN VAL -- return the unmasked value, exit policy here
    WHEN
        // The following code block determines whether the attribute passed to the policy should be subject
to masking or not
        // If it meets the following criteria then it is required that the attribute needs masking
        (
            // The security classification MUST be either 'Confidential' or 'Highly Confidential'
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code')
in ('Confidential', 'Highly Confidential')
            AND
            // At least one of the following classification flags must be 'Y'
            (
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
pci_dss_compliance_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag')
= 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
sensitive_personal_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag')
= 'Y'
            )
        )
    THEN '**MASKED**' -- return the masked value, exit policy here, the user DOES NOT have the data role to
allow unmasked data AND the attribute qualifies for masking
    ELSE VAL -- return the attribute value unmasked as none of the above is true and thus the attribute in
question does not require masking
END;

-- for Date datatype the mask is to present the attribute's value as '0001-01-01'
CREATE OR REPLACE MASKING POLICY SENSITIVE_DATA_MASK_DATE AS (VAL date) RETURNS Date ->
CASE
    WHEN
        // Check to see if the role(s) which has been identified as the one which ALLOWS the data attribute
to be presented UNMASKED
        // Replace 'RL_DAS_DEV02_CONSUMER' with the actual role (TBD within the uplift/data product streams)
        IS_ROLE_IN_SESSION('RL_DAS_DEV02_CONSUMER')
        THEN VAL -- return the unmasked value, exit policy here
    WHEN
        // The following code block determines whether the attribute passed to the policy should be subject
to masking or not
        // If it meets the following criteria then it is required that the attribute needs masking
        (
            // The security classification MUST be either 'Confidential' or 'Highly Confidential'
```

```
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code')
in ('Confidential', 'Highly Confidential')
            AND
            // At least one of the following classification flags must be 'Y'
            (
            SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
pci_dss_compliance_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag')
= 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.
sensitive_personal_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag') = 'Y'
            OR SYSTEM$GET_TAG_ON_CURRENT_COLUMN('DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag')
= 'Y'
            )
        )
        THEN '0001-01-01' -- return the masked value, exit policy here, the user DOES NOT have the data
role to allow unmasked data AND the attribute qualifies for masking
    ELSE VAL -- return the attribute value unmasked as none of the above is true and thus the attribute in
question does not require masking
END;
```

STEP 3 - CREATE THE REQUIRED SNOWFLAKE TAG REQUIRED TO ALIGN THE ATTRIBUTES DATATYPE TO IT'S RELEVANT MASKING POLICY

We are implementing our masking policies to product a relevant mask which aligns to the attributes datatype. As such we create a single tag and later assign it to three different masking policies as we are testing the mask for three datatypes:

- String
- Number
- Date

Create Tag to align to masking policies for Datatype

```
-- Create tag to anchor to the masking policies for the attribute's datatype
CREATE OR REPLACE TAG DB_CUSTOMER_DEV02.GOVERNANCE.datatype;
```

STEP 4 - APPLY MASKING POLICIES TO DATATYPE TAG

Associate the Datatype tag with the three masking policies

Apply masking policies to datatype tag

```
-- Associate the Datatype tag with the three masking policies
ALTER TAG DB_CUSTOMER_DEV02.GOVERNANCE.datatype SET MASKING POLICY DB_CUSTOMER_DEV02.GOVERNANCE.
SENSITIVE_DATA_MASK_STRING;
ALTER TAG DB_CUSTOMER_DEV02.GOVERNANCE.datatype SET MASKING POLICY DB_CUSTOMER_DEV02.GOVERNANCE.
SENSITIVE_DATA_MASK_NUMBER;
ALTER TAG DB_CUSTOMER_DEV02.GOVERNANCE.datatype SET MASKING POLICY DB_CUSTOMER_DEV02.GOVERNANCE.
SENSITIVE_DATA_MASK_DATE;
```

STEP 5 - CREATE THE REQUIRED SNOWFLAKE TAG REQUIRED TO ALIGN THE ATTRIBUTES DATATYPE TO IT'S RELEVANT MASKING POLICY

CREATE TEST SCENARIO

We use the table DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST as it has a good mix of DATE, STRING (alias VARCHAR) and NUMBER datatypes to test.

TEST POINT PRIOR TO MASKING THE TABLE (NOTE: THIS IS TEST DATA, NOT ACTUAL CUSTOMER DATA)

At this point no masking is applied, a select from the table using the following 7 columns:

- BIRTH_DATE
- CUSTOMER_ID
- FORENAME_1_NAME
- FORENAME_2_NAME
- SURNAME_NAME
- PASSPORT_EXPIRY_DATE
- PRIMARY_IDENTIFICATION_ID

The following code:

Unmasked select statement

```
SELECT TOP 100
BIRTH_DATE
, CUSTOMER_ID
, FORENAME_1_NAME
, FORENAME_2_NAME
, SURNAME_NAME
, PASSPORT_EXPIRY_DATE
, PRIMARY_IDENTIFICATION_ID
FROM DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST
WHERE DRIVER_LICENCE_VERSION_ID IS NOT NULL
```

Yields the attributes unmasked:

	BIRTH_DATE	CUSTOMER_ID	FORENAME_1_NAME	FORENAME_2_NAME	SURNAME_NAME	PASSPORT_EXPIRY_DATE	PRIMARY_IDENTIFICATION_ID
1	1980-01-01	1235193049	LINKED	PPPPPPP	Warner	null	qw123456
2	1979-06-07	1235118460	Thirhinethute	Rhiwo	Mihoqatogo	null	EH321194
3	2001-01-01	1235052430	Profile		Customer	2031-01-06	AU233564
4	2011-09-03	1236824068	Thirhinethute	Rhiwo	Mihoqatogo	null	YV5dZuyu4K
5	2011-09-03	1236824068	Thirhinethute	Rhiwo	Mihoqatogo	null	YV5dZuyu4K
6	1999-09-19	1235084432	Gerald	Jim	Saftary	2031-01-06	wwq
7	1999-09-19	1235084432	Gerald	Jim	Saftary	2031-01-06	wwq
8	1999-09-19	1235291095	Gerald	Jim	Saftary	2031-01-06	wwq
9	1999-09-19	1235291095	Gerald	Jim	Saftary	2031-01-06	wwq
10	1983-11-05	1236238819	pFPloMB upTzcl		oMFus	null	Ge2Mlv
11	1983-11-05	1236238819	pFPloMB upTzcl		oMFus	null	Ge2Mlv
12	1983-11-05	1236238819	pFPloMB upTzcl		oMFus	null	Ge2Mlv
13	1980-01-01	1234738609	Kiritaki	PPPPPPP	Ward	null	qw123456
14	1999-09-19	1235403245	Gerald	Jim	Saftary	2031-01-06	wwq
15	1999-09-19	1235403245	Gerald	Jim	Saftary	2031-01-06	wwq
16	1972-02-28	1236757803	ynDolxv xnxlYp		SlExp	null	Gi1YBc
17	1972-02-28	1236757803	ynDolxv xnxlYp		SlExp	null	Gi1YBc
18	1972-02-28	1236757803	ynDolxv xnxlYp		SlExp	null	Gi1YBc
19	1972-02-28	1236757803	ynDolxv xnxlYp		SlExp	null	Gi1YBc
20	1999-09-19	1236783438	Gerald	Jim	Saftary	2031-01-06	wwq

In our test scenario now the following same 7 columns in our dataset will be classified as being sensitive with the following classifications:

- BIRTH_DATE (DATE) - Classified as (security_classification_code = 'Confidential', sensitive_personal_identifier_flag = 'Y')
- CUSTOMER_ID (NUMBER)- Classified as (security_classification_code = 'Confidential', privacy_identifier_flag = 'Y')
- FORENAME_1_NAME (VARCHAR(30)) - Classified as (security_classification_code = 'Confidential', sensitive_personal_identifier_flag = 'Y')
- FORENAME_2_NAME (VARCHAR(30))- Classified as (security_classification_code = 'Confidential', sensitive_personal_identifier_flag = 'Y')
- SURNAME_NAME (VARCHAR(30))- Classified as (security_classification_code = 'Confidential', sensitive_personal_identifier_flag = 'Y')
- PASSPORT_EXPIRY_DATE (DATE)- Classified as (security_classification_code = 'Highly Confidential', government_identifier_flag = 'Y')
- PRIMARY_IDENTIFICATION_ID (VARCHAR(20))- Classified as (security_classification_code = 'Highly Confidential', government_identifier_flag = 'Y')

We will also apply the 'datatype' tag which is what will be used to 'call' the appropriate MASKING policy

Apply tags to table attributes

```
-- set 7 columns on the DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST table to implement
the relevant sensitivity classification tags.
-- the columns are:
---- BIRTH_DATE (DATE)
---- CUSTOMER_ID (NUMBER)
---- FORENAME_1_NAME (VARCHAR(30))
---- FORENAME_2_NAME (VARCHAR(30))
```

```
---- SURNAME_NAME (VARCHAR(30))
---- PASSPORT_EXPIRY_DATE (DATE)
---- PRIMARY_IDENTIFICATION_ID (VARCHAR(20))
-- we also apply the 'datatype' tag which is what will be used to 'call' the appropriate MASKING policy

-- we set this tag at the table level so it is automatically INHERITED at the column level in the table
-- note it can also be set at the column level directly
-- https://docs.snowflake.com/en/user-guide/object-tagging#tag-lineage
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.datatype = '';

// set BIRTH_DATE classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN BIRTH_DATE SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set CUSTOMER_ID classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN CUSTOMER_ID SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set FORENAME_1_NAME classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN FORENAME_1_NAME SET
TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set FORENAME_2_NAME classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN FORENAME_2_NAME SET
TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set SURNAME_NAME classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN SURNAME_NAME SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set PASSPORT_EXPIRY_DATE classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN
PASSPORT_EXPIRY_DATE SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Highly Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'N'
```

```
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';

// set PRIMARY_IDENTIFICATION_ID classification sensitivity flags
ALTER TABLE DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST MODIFY COLUMN
PRIMARY_IDENTIFICATION_ID SET TAG
DB_CUSTOMER_DEV02.GOVERNANCE.security_classification_code = 'Highly Confidential'
, DB_CUSTOMER_DEV02.GOVERNANCE.privacy_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.pci_dss_compliance_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.government_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.sensitive_personal_identifier_flag = 'Y'
, DB_CUSTOMER_DEV02.GOVERNANCE.health_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credit_identifier_flag = 'N'
, DB_CUSTOMER_DEV02.GOVERNANCE.credentials_identifer_flag = 'N';
```

We do not have the role RL_DAS_DEV02_CONSUMER applied to our user, so the policy code should return masked values for the 7 columns updated.

Now the following same select statement:

Masked select code

```
SELECT TOP 100
BIRTH_DATE
, CUSTOMER_ID
, FORENAME_1_NAME
, FORENAME_2_NAME
, SURNAME_NAME
, PASSPORT_EXPIRY_DATE
, PRIMARY_IDENTIFICATION_ID
FROM DB_CUSTOMER_DEV02.FDP_CUSTOMER_STORE.STORE_FDP_BST_INDIV_CUST
WHERE DRIVER_LICENCE_VERSION_ID IS NOT NULL
```

This correctly produces masked data.

	BIRTH_DATE	CUSTOMER_ID	FORENAME_1_NAME	FORENAME_2_NAME	SURNAME_NAME	PASSPORT_EXPIRY_DATE	PRIMARY_IDENTIFICATION_ID
1	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
2	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
3	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
4	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
5	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
6	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
7	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
8	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
9	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
10	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
11	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
12	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
13	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
14	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
15	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
16	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
17	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
18	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
19	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**
20	0001-01-01	-1	**MASKED**	**MASKED**	**MASKED**	0001-01-01	**MASKED**